Amp Server™ Pro SDK 3.0

For Scientific Research Purposes

NOT FOR CLINICAL OR DIAGNOSTIC USE

Network Protocols, Data Formats, and Technical Details

Programmer's Reference Manual

8200031-01

BEFORE USING THIS PRODUCT, READ, UNDERSTAND, AND AGREE TO ALL TERMS, DISCLAIMERS, AND LICENSING AGREEMENTS. The use of any Magstim Inc product (device or software) is subject to the terms of use, limits on resale or redistribution, the disclaimer of warranties, and the licensing agreement of that product. Users bear all risks of hazards, unexpected performance, or regulatory noncompliance associated with deviating from the supported configuration of Magstim Inc products or running Magstim Inc products with non-approved components.

Applicable Magstim Inc systems:

- The Geodesic Transcranial Electrical Neuromodulation (GTEN) 200 system, containing the Net Amps GTEN (NA GTEN) 200 amplifier, is a scientific research instrument.
- The GES 400 system may be converted to a scientific research instrument, allowing data communication between the Net Amps amplifier and a separate client computer.

  Only a qualified Magstim Inc representative performs this conversion. Once the system is converted, do not use this instrument in clinical applications or for clinical diagnosis. When GES 400 systems are configured for scientific research, external network connections are not supported. The end user is responsible for protecting the privacy of any personally identifiable information (PII) or protected health information (PHI) that may be stored on the instrument. The end user is responsible for reading and understanding all precautions and warnings supplied in the Amp Server Pro SDK Users Guide or license agreement.

Amp Server Pro SDK is a software development kit (SDK) library that allows users to capture data in real time from Magstim Inc amplifiers (NA 400 and NA GTEN 200) into non-Magstim Inc applications. Amp Server Pro SDK provides two methods for communicating with Amp Server:

1. Using a high-level API, which is a client-side protocol that is usable with languages that are linkable to C++ via its ready-to-use client libraries.

   The high-level (client-side protocol) API includes library support for Linux and Mac OS. These libraries have been tested under Mac OS 10.14.4 and Ubuntu 18.04.

2. Using a network protocol that is language independent.

WARNING: All system components must remain as integrated by a Magstim Inc support or authorized engineer. Deviating from the supported configuration by running the system with non-approved components attached, or repositioned components, may result in:

- hazards or unexpected performance due to additional loading or leakage; or
- poor system timing (and thus poor data quality) that may not be apparent.

CAUTION: Anyone who is not completely proficient with all aspects of customizing their EEG system should not attempt it. Poorly programmed applications could cause processes to fail or corrupt EEG data.

This document itemizes the commands that are the basis of the network protocols.

- Amplifier and Amp Server commands … page 2
- Command responses … page 7
- ECI commands … page 8
- Reading the data stream … page 15
- Scaling factors … page 20
- End-user software license … page 38
- GTEN 200 … page 22
  - MCF
  - Structure
  - Elements
  - Sending GTEN Trains
  - Controlling Neuromodulation
  - GTEN Notifications
  - Example
  - Monitoring TR
  - Compliance Voltage
  - Source/Sink Aggregate Current
  - Arbitrary Waveform Frequency

Amplifier and Amp Server Commands

All commands apply to NA 400 and NA GTEN 200 amplifiers.

Command	Arguments	Description
Amplifier		
cmd_None	int64_t amp id, int16_t channel (N/A), int16_t value (N/A)	This is a null command. It has no function and is not used to command the amplifier.
cmd_Start	int64_t amp id, int16_t channel (N/A), int16_t value (N/A)	This command causes the amplifier to begin to send data. The cmd_SetPower command should be used to turn on the amplifier before trying to issue the start command.
cmd_Stop	int64_t amp id, int16_t channel (N/A), int16_t value (N/A)	This command tells the amplifier to stop sending data.
cmd_TurnAll10KOhms	int64_t amp id, int16_t channel (N/A), int16_t value (0 = Off, 1 = On)	This command instructs the amplifier to place or remove 10 KOhm resistors between the inputs of all channels and ground. This state is used to measure impedance and noise.
cmd_TurnChannel10KOhms	int64_t amp id, int16_t channel (channels 0-287), int16_t value (0 = Off, 1 = On)	This command instructs the amplifier to place or remove 10 KOhm resistors between the inputs of a selected channel and ground. This state is used to measure impedance and noise.
cmd_SetCOM10KOhms	int64_t amp id, int16_t channel (N/A), int16_t value (N/A)	Like the cmd_TurnChannel10KOhms command, this command places a 10 KOhm resistor on the input to the COM channel. This can be used to measure impedance and noise of the COM channel.
cmd_TurnAllDriveSignals	int64_t amp id, int16_t channel (N/A), int16_t value (0 = Off, 1 = On)	This command instructs the amplifier to drive all channels with a test or calibration signal. The signal drives the inputs to the amplifier and provides a test signal that can be used in the absence of an EEG Net to test inputs to the amplifier. This command is used to measure impedance and gains.
cmd_SetCOMDriveSignal	int64_t amp id, int16_t channel (N/A), int16_t value (N/A)	Like the cmd_TurnAllDriveSignals command, this command turns on the calibrations signal on the input to the COM channel. This can be used for debugging and diagnostics for measuring impedance and gains.
cmd_TurnChannelDriveSignals	int64_t amp id, int16_t channel (channels 0-287), int16_t value (0 = Off, 1 = On)	This command instructs the amplifier to drive a selected channel with a test or calibration signal. The signal drives the inputs to the amplifier and provides a test signal that can be used in the absence of an EEG Net to test inputs to the amplifier. This command is used to measure impedance and gains.
cmd_SetSubjectGround	int64_t amp id, int16_t channel (N/A), int16_t state (0 = Off, 1 = On)	This command turns the common channel On and Off and is used only for impedance measurements.
cmd_SetCurrentSource	int64_t amp id, int16_t channel (N/A), int16_t value	This command switches the calibration signal between constant voltage and constant current. The default is constant voltage.
cmd_SetCalibrationSignalFreq	int64_t amp id, int16_t channel (N/A), int16_t value (frequency)	This command sets the frequency of the calibration signal that is enabled when the *cmd_TurnChannelDriveSignals* and *cmd_SetWaveShape* commands are used.
cmd_SetBufferedReference	int64_t amp id, int16_t channel (N/A), int16_t value (0 = On, 1 = Off)	This command enables the reference signal. It causes the reference electrode to become active and is used when measuring differential inputs on the channels.

Command	Arguments	Description
cmd_SetOscillatorGate	int64_t amp id, int16_t channel (N/A), int16_t value	This command disconnects the calibration signal from the switches used to introduce the signal on a channel or channels. When the calibration signal is not in use, having that signal on the open switch can still induce signals on the channels. This disconnects the signal well before the amplifier inputs to eliminate any induced signals in the data.
cmd_SetReference10KOhms	int64_t amp id, int16_t channel (N/A), int16_t value	Like the *cmd_TurnChannel10KOhms* command, this command places a 10 KOhm resistor on the input to the reference channel. This can be used to measure impedance and noise of the reference channel.
cmd_SetReferenceDriveSignal	int64_t amp id, int16_t channel (N/A), int16_t value	Like the *cmd_TurnAllDriveSignals* command, this command turns on the calibrations signal on the input to the reference channel. This can be used for debugging and diagnostics for measuring impedance and gains.
cmd_SetPower	int64_t amp id, int16_t channel (N/A), int16_t (0 = Off, 1 = On)	This command turns On and Off the amplifier.
cmd_Reset	int64_t amp id, int16_t channel (N/A), int16_t value (N/A)	This command resets the selected amplifier.
cmd_SetWaveShape	int64_t amp id, int16_t channel (N/A), int16_t (0 = Off (default sine wave, 1 = square wave, 2 = triangle wave, 3 = saw tooth)	This command determines the waveform of the calibration signal. By default, a sine wave is used.
cmd_SetDrivenCommon	int64_t amp id, int16_t channel (N/A), int16_t state (0 = Off, 1 = On)	This command is intended for use with CANCL technology. This provides for active cancellation of noise using the signal from the reference channel. When activated, the common channel is driven with the signal from the reference to actively cancel the noise signal on the normal EEG channels. This capability is only used in conjunction with EEG Nets that support CANCL technology.
cmd_ SetCalibrationSignalAmplitude	int64_t amp id, int16_t channel (set by default to channel 8), int16_t value (12 bit range)	This command sets the amplitude of the calibration signal.
cmd_SetDigitalOutputData	int64_t amp id, int16_t channel (N/A), int16_t value (bit mask of DIO line to be set)	This command sets the bits of the 16-bit Digital I/O port on the amplifier. By default, all DIO bits are intended for input, and this command will have no effect. To set specific bits to outputs bits, use the *cmd_SetDigitalInOutDirection* command.

Command	Arguments	Description
cmd_SetDigitalInOutDirection	int64_t amp id, int16_t channel (N/A), int16_t value (bit mask indicating the direction status of the I/O lines)	This command is to tell the amplifier to use specific bits of its 16-bit DIO board for input. By default, all DIO bits are intended for input. Their default behavior can be changed using this command. For example, to set the first 8 bits for input and the last 8 bits for output, send the binary value (1111111100000000) = 0xFF00 = 65280 in this command. The amplifier masks input bits when writing to output bits, so for this example, if a value is sent using the *cmd_SetDigitalOutputData* command that has bits set in the first byte (e.g., 00000010 01010110), these will not affect bits that are set for input. In this example, if all bits were zero before this call, the bits when read will be 00000000 01010110. Note that the second bit in the first byte is not set to one. A bit set to input is not affected by a *cmd_SetDigitalOutputData* command. Likewise, the value of a bit set to output is not affected by a signal on the port.
cmd_GetStartTime	int64_t amp id, int16_t channel (N/A), int16_t value (N/A)	This command gets the start time of the selected amplifier. This value is set every time the amplifier is started, and represents the system time when the amplifier started, measured in microseconds that have passed since the Epoch: 00:00:00 January 1, 1970
cmd_GetCurrentTime	int64_t amp id, int16_t channel (N/A), int64_t value (N/A)	This command is not supported.
cmd_GetCurrentDrift	int64_t amp id, int16_t channel (N/A), int64_t value (N/A)	This command is not supported.
cmd_SetNativeRate	int64_t amp id, int16_t channel (N/A), int16_t value	This command is similar to cmd_SetDecimatedRate, except that for the NA 400 and NA 410 you are getting the actual native sampling rate of the amplifier. The sample rate of the ADC that collects DIN input data is locked at 1,000 samples per second. This means that if you use cmd_SetNativeRate to specify a data collection rate for EEG of greater than 1,000 samples per second, the relationship between the timing of the EEG and the DIN data is no longer 1 to 1. **For NA 400s**: The values are 500, 1000, 2000, 4000, and 8000. **For NA 410s**: Only the native rate of 20000 is supported. Note that there is no software filtering at these rates, just a hardware filter on the sampling circuits. Once this command is sent, a subsequent call to cmd_SetDecimateRate will negate it.
cmd_SetDecimatedRate	int64_t amp id, int16_t channel (N/A), int64_t value	This command sets the decimation rate or downsampling rate of the amplifier's native rate to specific values from that amplifier's set: • ***NA 400s***: 250, 500, 1000
cmd_SetMRIPulseInfo	int16_t channel, int64_t value (high byte = DIN line (0 = detection off), low byte = DIN value)	This command is only used for MRI compatible amplifiers and is not supported in the Amp Server Pro SDK.
cmd_setPIBChannelGain	int64_t amp id, int16_t channel (1-32), int16_t value (NA 400/Physio16 gain = 1, 2, 3, 4, 6, 8, and 12; NA 300/PIB gain = 2, 20, 200, and 2,000)	Sets the gain of the named Physio16 (used with NA 400s) or PIB (used with NA 300s) channel to the specified value. Note: A gain of 20 matches the gain of EEG channels.

Command	Arguments	Description
cmd_TurnChannelZeroOhms	int64_t amp id, int16_t channel (0-287), int16_t value(0 = normal operation, 1 = ground)	If the value is 1, this command shorts the specified channel to ground. If the value is 0, this command sets the specified channel to operate normally. All other values are ignored.
cmd_TurnAllZeroOhms	int64_t amp id, int16_t channel (N/A), int16_t value(0 = normal operation, 1 = ground)	Like the cmd_TurnChannelZeroOhms command, this command shorts all channels to ground if the value is 1, or normal if the value is 0. All other values are ignored.
cmd_SetPhoticStimSequence	int64_t amp id, int16_t channel (N/A), char * value (mff PhoticStimRun xml string)	The MFF layer defines an XML format for PhoticStim sequences. Sending this command with a string composed of an xml PhoticStim sequence as the value will cause the NA 400 to parse the string and begin issuing PhoticStim pulses as specified.
cmd_ GetPhysioConnectionStatus	int64_t amp id, int16_t channel (N/A), int16_t value(N/A)	Issuing this command will cause the NA 400 to query the connection status of Physio16 hardware and issue a notification (over the notification port). Returned values can be interpreted as follows: 0: No Physio16 hardware is attached to the NA 400. 1: One Physio16 unit is attached on port 1. 2: One Physio16 unit is attached on port 2. 3: Two Physio16 units are attached.
Amplifier Status		
cmd_GetAmpDetails	int64_t amp id, int16_t channel (N/A), int64_t value (N/A)	This command returns details about the amplifier.
cmd_GetAmpStatus	int64_t amp id, int16_t channel (N/A), int64_t value (N/A)	This command returns general status information about the amplifier. It is currently not implemented.
Extended Amplifier Configuration Settings		
cmd_DefaultAcquisitionState	int64_t amp id (N/A), int16_t channel (N/A), int64_t value (N/A)	This is an 'extended' command that is received by Amp Server and is executed by sending the commands listed below to the amplifier. The default acquisition state is the normal state in which to place the amplifier for data acquisition. It turns off all components that are used for measuring gains, impedance, or noise and makes sure any calibration signal is off.
		cmd_TurnAll10KOhms, channel = 0, value = 0 cmd_TurnAllDriveSignals, 0, value = 0 cmd_SetSubjectGround, channel = 0, value = 1 cmd_SetCurrentSource, channel = 0, value = 0 cmd_SetCalibrationSignalFreq, channel = 0, value = 0 cmd_SetBufferedReference, channel = 0, value = 1 cmd_SetOscillatorGate, channel = 0, value = 0 cmd_SetReference10KOhms, channel = 0, value = 0 cmd_SetReferenceDriveSignal, channel = 0, value = 0 cmd_SetWaveShape, channel = 0, value = 0 cmd_SetCalibrationSignalAmplitude, channel = 0, value = 0 cmd_SetAnalogOutput, channel = 7, value = 0 cmd_SetDrivenCommon, channel = 0, value = 0
cmd_DefaultSignalGeneration	int64_t amp id (N/A), int16_t channel (N/A), int64_t value (N/A)	This is an 'extended' command that is received by Amp Server and is executed by sending the commands listed below to the amplifier. The default signal generation state is a typical state in which to place the amplifier for testing or debugging purposes. It enables the calibration signal and switches on the calibration signal for all channels.

Command	Arguments	Description
		cmd_TurnAllDriveSignals, channel = 0, value = 0 cmd_SetReferenceDriveSignal, channel = 0, value = 0 cmd_TurnAllDriveSignals, channel = 0, value = 1 cmd_SetSubjectGround, channel = 0, value = 0 cmd_SetCalibrationSignalFreq, channel = 0, value = 5 cmd_SetBufferedReference, channel = 0, value = 0 cmd_SetOscillatorGate, channel = 0, value = 1 cmd_SetCalibrationSignalAmplitude, channel = 0, value = 50 cmd_SetAnalogOutput, channel = 7, value = 50
Amp Server	Note: If a command is listed in AS_Network_Types_h, but not here, it is not supported in the SDK.	
cmd_NumberOfAmps	int64_t amp id (N/A), int16_t channel (N/A), int64_t value (N/A)	This command returns the number of amplifiers (active or not) seen by the system. *Active* usually means an amplifier is connected, but amplifiers can be *not active* for various reasons. To determine a connected amplifier's state, request its information.
cmd_NumberOfActiveAmps	int64_t amp id (N/A), int16_t channel (N/A), int64_t value (N/A)	Returns the number of amplifiers that are active in the system. *Active* usually means connected and available to stream data.
cmd_ListenToAmp	int64_t amp id, int16_t channel (N/A), int64_t value (N/A)	When connecting to the stream port, this command is sent to indicate the amplifier for which streaming is desired on this connection port.
cmd_StopListeningToAmp	int64_t amp id, int16_t channel (N/A), int64_t value (N/A)	This command is sent to stop listening to data from the amplifier on this port.
cmd_ReceiveNotifications	int64_t amp id (N/A), int16_t channel (N/A), int64_t value (N/A)	When connecting to the notification port, this command is sent to indicate the amplifier for which notifications are desired on this connection port.
cmd_ StopReceivingNotifications	int64_t amp id (N/A), int16_t channel (N/A), int64_t value (N/A)	This command is sent to stop receiving notifications from the amplifier on this port.
cmd_Exit	int64_t amp id (N/A), int16_t channel (N/A), int64_t value (N/A)	This command tells Amp Server to terminate.

Command Responses

Command responses will always be of the form:

(sendCommand_return …)

In addition, commands will always respond with status information relating to the command itself. This is indicated by the status element. Below is a list of valid status elements:

(status complete)

(status error)

Therefore, for a command that does not return any additional information, the two valid options are:

(sendCommand_return (status complete))

(sendCommand_return (status error))

Other commands return information. For example, the cmd_GetAmpDetails command returns:

(sendCommand_return (status complete) (amp_details (serial_number A14150128) (amp_type NA400) (legacy_board false) (packet_format 2) (system_version 1.6.15) (number_of_channels 256)))

Here are a few examples of additional information elements. Please refer to the AS_Network_Types.h header for an up-to-date list of supported commands and their return types.

(amp_server_status_info …)

(number_of_amps …)

(amp_status_info …)

(start_time …)

(current_time …)

(dio_values …)

Please note, the form (sendCommand_return …) is wrapped up in an AS_ReturnValue class if using the high-level API. For usage details, see the example code.

Experimental Control Interface (ECI) Commands and Return Values

as defined in AS_Network_Types.h

Specific Command	Controller Command (cmd) or Response	Follows controller command if more data expected	Description
ECI Commands			
eci_Query	**Q** controller cmd	cccc	This command is the legacy method for conveying machine type. It uses a four character code. Note that these are no longer descriptive of the actual machine type or byte order. NTEL, MAC+, etc., are not generally known to be big- or little-endian, but you should choose the appropriate one.
eci_NewQuery	**Y** controller cmd		This command is the new query for machine type and map specifically to the chip used by the OS. The return value should be one byte, and be the ECI version number.
eci_Exit	**X** controller cmd		This command ends the communications session. The server can assume that communications will no longer be sent after sending back the acknowledgement.
eci_BeginRecording	**B** controller cmd		This command tells the server application to start recording. The return value should be one byte.
eci_EndRecording	**E** controller cmd		This tells the server application to stop recording. The return value should be one byte.
eci_Attention	**A** controller cmd		This command is sent in advance of the chosen sync command. It provides the server indication that the sync command is about to be sent so it knows not to change anything on the server side that might affect synchronization.
eci_ClockSynch	**T** controller cmd	1111	This is the non-NTP synchronization command. Sending this command will result in the server recalculating the offset between clocks. This enables the client and server to account for the drift between the systems.
eci_NTPClockSynch	**N** controller cmd	1111	This command indicates that the client is also an NTP client and has a clock that is synchronized with the server clock (and in the case of Net Station, it is synchronized with the EEG being recorded. The time sent represents the absolute start time of the clients clock in NTP time format. The server will compare this value with the absolute time of start of its clock to determine the offset between clocks. Sending this command more than once will have no effect as the start times of each clock does not change. The return should be one byte.
eci_NTPReturnClockSynch	**S** controller cmd		This command is identical to the eci_NTPClockSynch command except that the server returns the start time of its clock in NTP form as well. This allows the client to adjust its clock or timestamps to already be relative to the server's clock sparing the server the responsibility to adjust incoming timestamps in events. The return value should be 8 byte NTP time from server.

Specific Command	Controller Command (cmd) or Response	Follows controller command if more data expected	Description
eci_EventData	**D** controller cmd	<data>	This command sends data as a timestamp event. In all cases, timestamps will be 32 bit integers representing relative time from some absolute start time. For normal (eci_ClockSynch) and NTP (eci_NTPClockSynch) sync methods, the timestamps for these events will be relative the start time of the client's clock. For NTP with return time (eci_NTPReturnClockSynch), these relative times will already be converted so they are relative to the servers clock. The return value should be one byte.
ECI Return Values			
eci_OK	**Z** response		One byte return value equal to 'Z'.
eci_Failure	**F** response		One byte return value equal to 'F'.
eci_NoRecordingDeviceFailure	**R** response		One byte return value equal to 'R'.
eci_Identify	**I** response		One byte return value equal to 'I'.

Experiment Control Protocol

The following aids you in your design of a custom external controller.

Requirements

Hardware Requirements

A TCP/IP connection is required to link the Net Station EEG data acquisition computer and the experimental control computer (ECC). Net Station must run on a Mac computer, but the machine providing experiment control need not be a Mac.

Whatever the experiment control machine, it must communicate to Net Station via TCP/IP.

For TCP/IP port communication, you can connect to ports 55513, 55514, 55515, or 55516—port 55513 is preferred.

During the initial query, you should send the machine "type" of the ECC, if using the legacy query command. Type in this case means byte order. For example, Intel processors are typically little-endian; therefore, you would send the following during a query exchange:

'QNTEL'

- Q for the query command
- NTEL for the machine type

You may use either the new or legacy query command, as follows.

```
// ECI machine types.
static const ECIMachineType  eci_unknown =     'u';
static const ECIMachineType  eci_i386 =        'i';
static const ECIMachineType  eci_x86_64 =      'x';
static const ECIMachineType  eci_ppc =         'p';


/**
        *  Legacy ECI machine types.
    *
    *  Notes:
    *
    *  1) Given that all clients are distributing these fourcc types as
    *  simple char strings,
    *  is is probably better to move these to char strings in the future.
        */
   typedef enum {

      eciMachineType_Mac              = 'MAC-',        // Big-endian
      eciMachineType_Unix             = 'UNIX',        // Big-endian
      eciMachineType_Intel       = 'NTEL'         // Little-endian


   } LegacyECIMachineType;
```

Note: *Little-endian* refers to a method of storing two- and four-byte integers in which the lower address has the lower significance, or "little end first." Most processors, other than Intel machines and VAXen, use *big-endian* notation, which means that the most significant byte, or "big end," is stored in the lower address.

Software Requirements

Amp Server Pro SDK 3.0 works within the following software environments:

- Net Station 5.4.3-R
- A server-side environment that supports the ECI protocol

Protocol Description

The following serves as more detailed descriptions of the usage of the commands and types listed in the AS_Network_Types.h header file. There is some repetition for the sake of clarity.

These commands are designed to reduce communication latency by minimizing the amount of data transmitted.

Commands

All commands consist of a single ASCII character followed by a zero or additional bytes of information. Additional information, except in the case of the **D** command, is expected to be one of four data types.

Data type descriptions

Data type	Description
cccc	A four-character descriptor type (DescType); usually printable ASCII characters
b	A single byte interpreted as a number between -128 and 127
ss*	A short integer (2 bytes) representing a number between -32,768 and 32,767
llll*	A long integer (4 bytes) representing a number between -2,147,483,648 and 2,147,483,647

*Numbers represented by ss and llll are machine dependent.

Command Summary

Commands, responses, and descriptions

Controller command	Brief description	Net Station responses	Brief description
Qcccc	Hello, I am … (machine type)	**IB** or **Fss**	Identify/I'm version … or Failure
X	Exit	**Z** or **Fss**	OK or Failure
B	Begin recording	**Z** or **Fss**	OK or Failure
E	End recording	**Z** or **Fss**	OK or Failure
A	Attention	**Z** or **Fss**	OK or Failure
TIIII	Clock sync	**Z** or **Fss**	OK or Failure
D<data>	Event data stream	**Z** or **Fss**	OK or Failure after complete transmission

Command Descriptions

Command **Q**cccc. Initiates a connection between the ECC computer and the EEG computer. The four-character descriptor type indicates which computer type is connected so that Net Station can account for any integer notation problems.

The descriptor must be one of the following:

'MAC-'	Macintosh computer
'UNIX'	UNIX computer
'NTEL'	Intel computer (i.e., a PC)

Command **X**. Indicates that the experiment has successfully ended. Net Station closes the file to which it has been sending data (the Recording file) and returns to an idle state.

Command **B**. Instructs Net Station to begin recording. If a Recording file has not been specified, then a new file is created using the default naming scheme. If the Acquisition Setup does not contain a device that belongs to the Recorder class, then Net Station returns an error.

Command **E**. Ends the recording but does not close the file. Net Station is capable of storing multiple recording epochs in a single file and does not impose a limit on the number of B and E sequences that it might receive per file.

Command **A**. Requests Net Station to do nothing else until another command is sent. Use the command to prepare Net Station for time synchronization. Once Net Station has successfully responsed to the A command, a T command is usually sent.

Command **T**1111. Synchronizes the ECC computer's time base with Net Station's time base. The long value is the ECC's current time in milliseconds.

Command **D**. Transmits event information. An event may be loosely defined as any occurrence that has both time and duration and needs to be recorded. Net Station provides a flexible and extensible structure for storing events for later retrieval and display. The transmission protocol for the data following this command is described in detail in the following section.

Note: To synchronize transmitted stimulus/response data, Net Station stores the ECC's time value, along with its own time value, at the moment of synchronization. When an event is received, Net Station subtracts the event time from the ECC's sync time and adds this to Net Station's time sync value. In other words, you should only send event times that are relative to the start of the recording in milliseconds.

Event Data Stream

The ECI device creates stimulus events by converting the information accompanying the D command into an event object. An event object is composed of several required fields, several optional fields, and an optional list of keyword-described data fields.

Event data stream fields and descriptions

Field	Required/Optional	Description
Start Time	Required	Event start time (in milliseconds)
Duration	Required	Event duration (in milliseconds). All events must have a duration of at least one millisecond.
Source Device	Required	Net Station stimulus device from which the event originated. This field is assigned by Net Station and cannot be programmed by an external controller.
Event Code	Required	A unique, user-defined, four-character descriptor type that identifies this event type. For instance, you might use the code 'AUDI' to indicate an audio stimulus.
Event Description	Optional	A string that even further describes the event.
Key List	Optional	A list of fields containing additional properties and data to further relate information about the event.



CAUTION: Do not send simultaneous ECI events (that is, ECI events with the same time stamps) to Net Station. Although Net Station can record simultaneous ECI events, users cannot access them properly because the events are on the same track. Simultaneous events on different tracks (for example, DIN events) can be accessed properly.

Event Key Field Descriptions

Event key data types are described and used by Net Station as follows. Other user-defined data types are not excluded or rejected by Net Station event-handling functions.

Data length. Length (in bytes) of the *variable length data* field.

Data type. A four-character descriptor type that describes the data type of the data stored in the Event Key. Net Station currently supports seven predefined descriptor types.

Key. A unique four-character identifier for the data.

Variable length data. The raw data.

Predefined descriptor types (codes have built-in meaning)

Type	**Length** (bytes)	**Description**
'bool'	1	Boolean (0 or 1)
'shor'	2	Short signed integer
'long'	4	Long signed integer
'sing'	4	Single-precision real
'doub'	8	Double-precision real
'type'	4	Four-character descriptor type
'TEXT'	variable	Sequence of characters

Note: Mac OS developers will recognize these descriptor types as Apple event descriptor types.

The data following the Data Event command are flattened and packed representations of the preceding event object structure. The entire Data Event command sequence is as follows.

Data Event command sequence

Type	**Length** (bytes)	**Description**
'D'	1	Data Event command
unsigned short	2	Length of following data block (bytes)
long	4	Event start time
long	4	Event duration
Desc	4	TypeEvent code
byte	1	Event label length, or 0 if no label
characters	variable	Event label (256 characters max.)
byte	1	Event description length, or 0 if no description
characters	variable	Event description (256 characters max.)
unsigned character	1	Number of keys, or 0 if no keys
DescType	4	Key
DescType	4	Data type
unsigned short	2	Length
??	variable	Variable length data

Response Descriptions

Response **I**b. Follows a successful query command by the controlling device. The single byte of data following the command indicates the version number of the protocol that is currently being supported.

Response **Z**. Follows any other successful command other than the initial query.

Response **F**ss. Is sent by Net Station when it receives an incorrectly constructed or undefined command, or is unable to comply with the command's instructions. No error numbers are currently defined.

Reading the Data Stream

CAUTION: Packet format is firmware version dependent. Before upgrading your amplifier's firmware, confirm compatibility with EGI Technical Support. Otherwise, an upgrade could break your code.

There are two packet formats, depending upon the Net Amps model being used and, in the case of the NA 400, the version of the firmware.

Amplifier Firmware	Packet Format 1	Packet Format 2
NA 400 or NA GTEN 200 *with 2.0.16 or later*		

Packet Format 2

As with Packet Format 1, you will receive 16 initial bytes, as the network protocols deliver data with Packet Format 2 in the following manner. Note the change to Packet Format 2.

Layout	| 8 Bytes (Network Byte Order)	| 8 Bytes (Network Byte Order)	| Sample Packets…	|
Description	| Amp ID	| Size of Data (in bytes)	| Sample Packets in Packet Format 2	|

*When using the high-level API, the data received in the AmpDataObject's update function has already removed the Amp ID and the size of the data fields. These fields are only present if you are working with the network layers directly. See the Simple Client in the examples for the high-level usage.

The data packets for Packet Format 2 are as follows:

```
#pragma pack(1)

    /**
    *  Packet Format 2 (Used NA400 in firmware versions AFTER 1.4.3.).
    *
    *  <b>Notes:</b><br>
    *    1) If this is raw data from the amplifier, it will be in analog to digital units
    *  and this will need to be converted to microvolts. Please see the Amp Server Pro SDK
    *  documentation for details.<br>
    *
    *    2) If this is raw data from the amplifier, it will be in analog to digital units
    *  and this will need to be converted to microvolts. Please see the Amp Server Pro SDK
    *  documentation for details.<br>
    *
    *    3) Data in this packet format is always in little endian byte order.<br>
    */
   typedef struct
   {
      uint8_t digitalInputs;
      uint8_t status;
      uint8_t batteryLevel[ 3 ];
      uint8_t temperature[ 3 ];
```

```
    uint8_t sp02;
    uint8_t heartRate[ 2 ];
} PacketFormat2_PIB_AUX;

typedef struct
{
    uint16_t            digitalInputs;
    uint8_t             tr;
    PacketFormat2_PIB_AUX   pib1_aux;
    PacketFormat2_PIB_AUX   pib2_aux;
    uint64_t            packetCounter;
    uint64_t            timeStamp;
    uint8_t             netCode;
    uint8_t             reserved[ 38 ];
    uint32_t            eegData[ 256 ];
    uint32_t            auxData[ 3 ];
    uint32_t            refMonitor;
    uint32_t            comMonitor;
    uint32_t            driveMonitor;
    uint32_t            diagnosticsChannel;
    uint32_t            currentSense;
    uint32_t            pib1_Data[ 16 ];
    uint32_t            pib2_Data[ 16 ];
} PacketFormat2_SamplePacket;

#pragma pack()
```

Pay special attention to the declaration of `#pragma pack(1)` ... `#pragma pack()` around the definition of the `PacketFormat2_SamplePacket`. These statements ensure that the compiler packs the members of the structure on 1-byte boundaries, precluding optimizations that might pad the length of members to align on some default byte length.

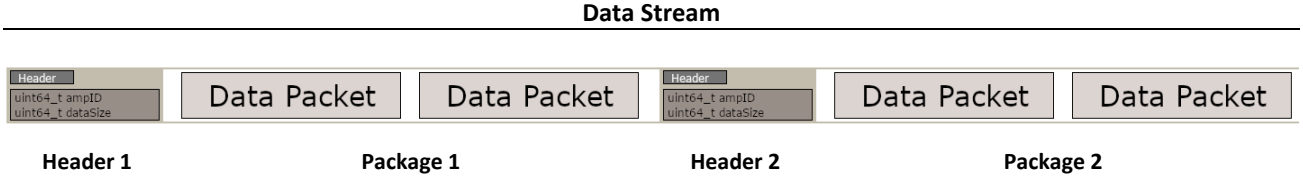
As the alignment of the data streamed from the amplifier is streamed aligned to 1-byte boundaries, it is essential that the structure used to represent this data is defined in the same way. Failure to properly define the structure will result in unusable data.

The size for each packet is 1264 bytes.

As with Packet Format 1, the order of operations on a coding level is to first read 16 bytes of "Amp ID" and "Size of Data", then read the "Size of Data" number of bytes from the network port. For example, suppose the following was sent.

| 0 | 6320 | Samples |

Your software would read the first 16 bytes. This will give an Amp ID of 0, and a size for the samples section of 6320. This means that you have received 6320 / 1264 = 5 samples laid out in Packet Format 2.



Sampling Rates and Net Types For Both Formats

Sample Rate. With the NA 400 and NA GTEN 200 amplifiers, the sampling rate can drop below 1000 samples per second.

This is accomplished using the following command:

" cmd_SetDecimatedRate"

For command details, see section "Amplifier and Amp Server Commands."

If the sampling rate drops below 1000 s/s, you will STILL receive data at 1000 s/s. This is for legacy reasons. Thus, if you drop the sampling rate to 250 s/s, the data will be "up-sampled" back to 1000 s/s by replicating. You will, in this case, receive four identical samples (almost), and thus you can discard three out of the four samples.

The "almost" is due to the digital inputs—the samples are truly identical.

It is possible to eliminate filtering and decimation of the NA 400 or the NA GTEN 200 amplifier. This mirrors the 1000 s/s TMS setting in Net Station Acquisition (v 5.4.3-R). By doing this, you can avoid the decimation filter delays at 1000 s/s. Be aware, however, that your effective analysis frequency range is one quarter of the sampling rate in this mode. Therefore, at 1 k samples per second, it is 1000/4 = 250 Hz. The command for this is: cmd_SetNativeRate.

If you select a sampling rate of less than 1000 s/s (i.e., 500), you will see repeated samples up to 1 k as with the cmd_SetDecimatedRate command.

Note that Net Station 5.4.3-R supports only the 1000 s/s TMS setting. Amp Server Pro SDK, however, provides more options.

For each amplifier's hardware anti-alias filter delay at different sampling rates, and for details regarding the 1000 s/s TMS sampling rate, refer to the Net Station 5.4.3-R manual (8100051).

Physio16 Research Data – Packet Format 2 Only

A good example of how to decode physiological aux information (PacketFormat2_PIB_AUX) is provided in the Simple Client example in the class DataStreamObserver, and should be referred to for more information. Up to two Physio16 Research input boxes are supported on a single NA 400 or NA GTEN 200 amplifier, as can be seen from the PacketFormat2_SamplePacket data structure.

There is an accepted mechanism (perhaps not ideal) for detecting the presence of a physiological unit, and this is exampled in the PhysioDetection class, again in the Simple Client example.

Net types (or codes as they are sometimes called):

```
// Net codes
typedef enum{
  GSN64_2_0,      //  GSN 64
  GSN128_2_0,     //  GSN 128
  GSN256_2_0,     //  GSN 256

  HCGSN32_1_0,    //  HGSN 32
```

```
  HCGSN64_1_0,    //  HGSN 64
  HCGSN128_1_0,   //  HGSN 128
  HCGSN256_1_0,   //  HGSN 256

  MCGSN32_1_0,    //  MGSN 32
  MCGSN64_1_0,    //  MGSN 64
  MCGSN128_1_0,   //  MGSN 128
  MCGSN256_1_0,   //  MGSN 256

  AMP_SAMPLE,     //  EAmpSample displayable channels (internal use only)
  TestConnector = 14,
  NoNet = 15,     //  net not connected
  Unknown = 0xFF  //  Unknown or net not connected

}NetCode;
```

Scaling Factors for NA 400s and NA GTEN 200s

In order to convert raw data to microvolts, scaling factors need to be applied to the raw streaming data. Which scaling factors are used is dependent upon the type of amplifier (NA 400 or NA GTEN 200) that is streaming the raw data and the specific data being converted. In general, multiply the raw value by the specific scaling factor (as indicated by the scaling factor tables below) to get the microvolts.

```
microvolts = raw_value x scale_factor
```

Note: All scaling factors assume the gains shown in the calculations on pages 20-21.

NA 400 and NA GTEN 200 Scaling Factors

Variable Name	NA 400 and NA GTEN 200 Scaling Factors	scale_factor
eegData [0 - 255]	NA 400 and NA GTEN 200 EEG Scale Factor	0.000155220429
auxData [0 - 2]	NA 400 and NA GTEN 200 EEG Scale Factor	0.000155220429
refMonitor	NA 400 and NA GTEN 200 EEG Scale Factor	0.000155220429
comMonitor	NA 400 and NA GTEN 200 EEG Scale Factor	0.000155220429
driveMonitor	NA 400 and NA GTEN 200 EEG Scale Factor	0.000155220429
diagnosticsChannel	NA 400 and NA GTEN 200 EEG Scale Factor	0.000155220429
pib1_Data [0 - 7]	Physio16 Scale 1-8	-0.00111758708
pib1_Data [8 - 15]	Physio16 Scale 9-16	0.00111758708
pib2_Data [0 - 7]	Physio16 Scale 1-8	-0.00111758708
pib2_Data [8 - 15]	Physio16 Scale 9-16	0.00111758708

Scaling Factors

NA 400 and NA GTEN 200 EEG Scale Factor. If the NA 400 amplifier has a legacy board (currently, only the NA 410), use the following scaling factor:

```
ref_voltage    = 4.096
amp_gain       = 19.7936

range_in_volts = ref_voltage * 2 / amp_gain
               = 4.096 * 2 / 19.7936
               = 0.41387115027

volts_per_bit  = range_in_volts >> 24
               = (ref_voltage * 2 / amp_gain) >> 24
               = 2.46686429e-8

mv_per_bit     = volts_per_bit * 1,000,000
               = ((ref_voltage * 2 / amp_gain) >> 24 ) * 1,000,000
               = 0.02466864289

eeg_scale      = mv_per_bit >> 8
               = (((ref_voltage * 2 / amp_gain) >> 24 ) * 1,000,000) >> 8
               = 0.00009636188
```

For all other NA 400 amplifiers, use the following scaling factor:

```
ref_voltage    = 4.0
amp_gain       = 12

range_in_volts = ref_voltage * 2 / amp_gain
               = 4.0 * 2 / 12
               = 0.666 . . . 7

volts_per_bit  = range_in_volts >> 24
               = (ref_voltage * 2 / amp_gain) >> 24
               = 3.973643e-8

mv_per_bit     = volts_per_bit * 1,000,000
               = ((ref_voltage * 2 / amp_gain) >> 24 ) * 1,000,000
               = 0.03973642985026

eeg_scale      = mv_per_bit >> 8
               = (((ref_voltage * 2 / amp_gain) >> 24 ) * 1,000,000) >> 8
               = 0.0001552204291
```

Physio16 Scale 1-8. For channels 1–8 of the Physio16-1 or Physio16-2, use the following scaling factor:

```
ref_voltage    = 2.4
amp_gain       = 1

range_in_volts = ref_voltage * 2 / amp_gain
               = 2.4 * 2 / 1
               = 4.8

volts_per_bit  = range_in_volts >> 24
               = (ref_voltage * 2 / amp_gain) >> 24
               = 2.86102295e-7
```

```
mv_per_bit      = volts_per_bit * 1,000,000
                = ((ref_voltage * 2 / amp_gain) >> 24 ) * 1,000,000
                = 0.28610229492

pib_scale_1_8   = -1 * mv_per_bit >> 8
                = -1 * (((ref_voltage * 2 / amp_gain) >> 24 ) * 1,000,000) >> 8
                = - 0.00111758708
```

Physio16 Scale 9-16. For channels 9–16 of the Physio16-1 or Physio16-2, use the following scaling factor:

```
ref_voltage     = 2.4
amp_gain        = 1

range_in_volts  = ref_voltage * 2 / amp_gain
                = 2.4 * 2 / 1
                = 4.8

volts_per_bit   = range_in_volts >> 24
                = (ref_voltage * 2 / amp_gain) >> 24
                = 2.86102295e-7

mv_per_bit      = volts_per_bit * 1,000,000
                = ((ref_voltage * 2 / amp_gain) >> 24 ) * 1,000,000
                = 0.28610229492

pib_scale_9_16  = mv_per_bit >> 8
                = (((ref_voltage * 2 / amp_gain) >> 24 ) * 1,000,000) >> 8
                = 0.00111758708
```

Support for GTEN 200

AmpServerProSDK 3.0 supports the programmatic control of GTEN 200 amplifiers. This control is based on a subset of the MCF XML specification.

MCF

MCF XML specifies the parameters for neuromodulation, including:

1. **Waveform**(s): The GTEN amplifier implements an arbitrary waveform generator and is capable of producing DC current for injection.
2. **Block**(s): Blocks, as defined by the MCF XML specification, specify target electrodes and the amplitude of the signal at those electrodes, as well as the time duration of the current injection and the index of the waveform to use for the injection.

Structure

The structure of MCF XML follows the general format listed below:

```
<?xml>
  <train>
    <waveforms>
      <{waveformType} id="{waveform_id}" />
    </waveforms>
    <blocks>
      <block id="{block_id}">
        <duration_ms>{block_duration}</duration_ms>
        <waveformId>{block_waveform_id}</waveformId>
        <spatialPattern>
          <electrode channelID="{source_electrode_channel}" amplitude="{source_electrode_amplitude}"/>
          <electrode channelID="{sink_electrode_channel}" amplitude="{sink_electrode_amplitude}"/>
        </spatialPattern>
      </block>
    </blocks>
  </train>
</xml>
```

Elements

Element/Parameter	Description
<train>	The train element is the root container element of the MCF format (ignoring the <?xml> tag). A train contains all of the necessary information to begin a neuromodulation session. The direct children of the train element are <waveforms> and <blocks>.
<waveforms>	The waveforms element is just a container for all of the waveforms specified for a given neuromodulation train. The number of waveforms specified is independent of the number of blocks, as many blocks can reference a single waveform or more waveforms than blocks can be specified (in which case some of those waveforms will not be used).
<dcWaveform>	The dcWaveform waveform type specifies that a simple DC level be used for neuromodulation. This waveform type has no parameters other than the id, which is

Element/Parameter	Description
	required to allow blocks to refer to it.
<pulsedDCWaveform>	The pulsedDCWaveform waveform type specifies a monophasic square wave wherein the duty cycle is defined by a ratio of ON period to OFF period. The pulsedDCWaveform, apart from id, has the following parameters:
<pulseLength_us>	The length of the ON period of the square wave in microseconds.
<delayLength_us>	The length of the OFF period of the square wave in microseconds.
<rampWaveform>	A rampWaveform represents an ascending or descending ramp. The rampWaveform, apart from id, has the following parameters:
<type>	Acceptable values are "up" for an ascending ramp and "down" for a descending ramp.
<duration_ms>	The duration of the ramp in milliseconds. Note that a periodic arbitrary signal generator generates the ramp, and if the duration of the ramp is not equal to the duration of the block using it, the behavior will be as follows:
The Duration of the Block Exceeds the Duration of the Ramp	A sawtooth pattern will be generated such that the length of the rising or falling edge of the sawtooth is defined by the duration of the rampWaveform and the number of teeth is determined by the dividend of the duration of the block divided by the duration of the rampWaveform.
The Duration of the Ramp Exceeds the Duration of the Block	A ramp will be generated, but will never reach the maximum or minimum level (depending on the ramp type).
<arbitraryWaveform>	An arbitraryWaveform is specified as a set of samples along with parameters defining how those samples should be played back to generate the desired waveform. The arbitraryWaveform, apart from id, has the following parameters:
<samples>	The samples should be specified as 1024 space delimitted floating point numbers in the range [-1.0, 1.0].
<frequencyNumerator>	Frequency should be specified as a ratio where frequencyNumerator defines the numerator of this ratio. This ratio will determine the duration of each sample.
<frequencyDenominator>	Frequency should be specified as a ratio where frequencyDenominator defines the denominator of this ratio. This ratio will determine the duration of each sample.
<blocks>	The blocks element is just a container for all of the blocks specified of a given neuromodulation train. The number of waveforms specified is independent from the number of blocks, as many blocks can reference a single waveform or more waveforms than blocks can be specified (in which case some of those waveforms will not be used).
<blocks>	A block controls discrete neuromodulation periods. A train can be composed of between 1 and 159 blocks, where each block specifies the target electrodes, the amplitude of the signal at each target, the waveform that should be used and the duration of the neuromodulation period. A block, apart from id, has the following parameters:
<duration_ms>	The length of time the block should be used to define the parameters of current injection in milliseconds. This should be specified as an integer.
<waveformId>	The ID of the waveform to use for current injection.

Element/Parameter	Description
<spatialPattern>	The spatialPattern element is a container for a listing of the electrodes into which current will be injected. It is important to note that the cathodal and anodal currents must be balanced to ensure that the system can deliver the requested current.
<electrode>	Each electrode element specifies a channel number for neuromodulation and the amplitude of the signal to be injected on the specified channel. The electrode has the following parameters: channelID The zero-indexed channel number to inject current at. amplitude The amplitude of the signal to inject at the specified channel. The acceptable range is [-200, 200] where the value specifies the current, in microamps, to deliver at the specified channel. The value of this amplitude value is used to scale the amplitude of the waveform, which is specified in normalized values.

Sending GTEN Neuromodulation Trains to the Amplifier

The GTEN amplifier implements a small set of new commands to facilitate the usage of the GTEN hardware. In order to control the complex capabilities of the hardware, the commands that allow the configuration of the neuromodulation trains accept MCF format XML strings.

Command	Description
cmd_GTENSetTrain	The cmd_GTENSetTrain command allows the user to set a complete GTEN Train, after which the amplifier will be prepared to start modulation. **Example:** (sendCommand cmd_GTENSetTrain 0 0 <?xml><train><waveforms><dcWaveform id="0" /> </waveforms><blocks><block id="0"><duration_ms>120000</duration_ms><waveformId>0</waveformId> <spatialPattern><electrode channelID="0" amplitude="50"/><electrode channelID="255" amplitude="-50"/></spatialPattern></block></blocks></train></xml>)
cmd_GTENSetWaveforms	The cmd_GTENSetWaveforms command allows the user to set the waveforms that will, later, be referred to by the waveformIds of any blocks that are sent to the amplifier. **Example:** (sendCommand cmd_GTENSetWaveforms 0 0 <?xml><waveforms><dcWaveform id="0" /></waveforms></xml>)
cmd_GTENSetBlocks	The cmd_GTENSetBlocks command allows the user to set the blocks that define the parameters of the neuromodulation train. **Example:** (sendCommand cmd_GTENSetBlocks 0 0 <?xml><blocks><block id="0"> <duration_ms>120000</duration_ms><waveformId>0</waveformId><spatialPattern> <electrode channelID="0" amplitude="50"/><electrode channelID="255" amplitude="-50"/></spatialPattern></block></blocks></xml>)

Controlling Neuromodulation

The system is armed once a neuromodulation train has been sent, and the train can be started at any time. Once the train has been started, it is possible to stop it at any time. Additionally, the current status of the GTEN system can be queried, and the overcurrent alarm can be reset through the AmpServerProSDK API.

Command	Description
cmd_GTENStartTrain	Assuming that a valid train has been sent, cmd_GTENStartTrain will immediately start neuromodulation.
cmd_GTENAbortTrain	Assuming that a train is currently running, cmd_GTENAbortTrain will immediately stop neuromodulation. There are no negative impacts of invoking cmd_GTENAbortTrain when no train is running.
cmd_GTENGetStatus	Invoking cmd_GTENGetStatus causes the amplifier to return information about the current status of the GTEN hardware. This information is returned over the notification port.

GTEN Notifications

Notification	Description
ntn_GTENGetStatus	
Firmware Version	The GTEN microcontroller firmware version.
Remaining Time(ms)	The time remaining in the most recently specified train. This will report the full duration of the train if it is not running, but the actual time remaining if the train is currently running.
Sentinal Alarm	The status of the overcurrent alarm is reported as either Off or On.
Serial Number	The serial number of the GTEN microcontroller.
Train Running	The status of the run state of the GTEN Train is reported as either Yes or No.
Waveform Pulsed	Specifies whether the waveform in use is pulsed
Waveform Type	Specifies the type of waveform: either AC or DC.
ntn_GTEN<X>Train<X>	
ntn_GTENSetTrainSucceeded / ntn_GTENSetTrainFailed	ntn_GTENSetTrainSucceeded is returned if the system is successful configuring the GTEN subsystem based on the previously specified GTEN Train. If setting the train fails, ntn_GTENSetTrainFailed will be returned instead.
ntn_GTENSetWaveformSucceeded	For each waveform successfully sent to the GTEN subsystem, an ntn_GTENSetWaveformSucceeded is returned. Once the number of notifications is equal to the number of waveforms sent, it is safe to assume that all waveforms

Notification	Description
	have been set. If this equality is not reached, it indicates that not all waveforms have been successfully interpreted by the system.
ntn_GTENStartTrainSucceeded / ntn_GTENStartTrainFailed	When the cmd_GTENStartTrain command succeeds, an ntn_GTENStartTrainSucceeded notification will be returned over the notification port. If starting the train fails, ntn_GTENStartTrainFailed will be returned instead.
ntn_GTENAbortTrainSucceeded / ntn_GTENAbortTrainFailed	When the cmd_GTENAbortTrain command succeeds, a ntn_GTENAbortTrainSucceeded notification will be returned over the notification port. If stopping the train fails, ntn_GTENAbortTrainFailed will be returned instead.
ntn_GTENResetAlarmSucceeded / ntn_GTENResetAlarmFailed	When the cmd_GTENResetAlarm command succeeds, an ntn_GTENResetAlarmSucceeded notification will be returned over the notification port and, if the overcurrent alarm is sounding, the alarm will stop. If resetting the alarm fails, ntn_GTENResetAlarmFailed will be returned instead and, if the overcurrent alarm is sounding, the alarm will not stop and the system will continue to prevent further neuromodulation.

Example

The following demonstrates the usage of the GTEN-specific commands using a tool such as Netcat (nc) and the low-level interface. The GTEN command interface is a part of the larger command interface and does not affect the remainder of the system, including the stream interface of the amplifier.

Feature/Function	Command/Description
Prerequisites	Assuming the use of Netcat, the following examples can be performed by opening two terminal windows. For these examples, we will ignore the stream interface, as interpretation of the data stream requires additional work described elsewhere.
Notification Port	The notification port can be connected to by running the following command in a terminal window: `nc 10.10.10.51 9878` *10.10.10.51 should be replaced with the IP address of the NA GTEN 200 amplifier.* Once this command has been entered, the cursor will move to the next line and print something similar to the following: `(sendCommand_return (status complete) (amp_server_status_info (client_connected (client_id 19))))` At this point, the notification port is connected and any notifications produced should be printed in this window.

Feature/Function	Command/Description

Command Port	The command port can be connected to by running the following command in a terminal window:
	`nc 10.10.10.51 9877` *10.10.10.51 should be replaced with the IP address of the NA GTEN 200 amplifier.*
	Once this command has been entered, the cursor will move to the next line and wait for user input. To initialize the amplifier enter the following in the command terminal:
	`(sendCommand cmd_SetPower 0 0 1)`
	`(sendCommand cmd_Start 0 0 1)`

Using GTEN with the SDK	Using the GTEN hardware involves first configuring, then starting a neuromodulation train. There are two pieces of information that must be specified before current injection can be initiated: • the waveform to use • the sensors to target There are two ways to specify this information: • Specify both in a GTEN Train. • Upload the waveforms and the GTEN Blocks separately. *Often preferable.* The GTEN Blocks define the duration and the target sensors. Once the train is running, one can allow it to complete or abort it. Completion entails waiting a duration of time equal to the sum of the durations of the blocks that make up the train.
Sending a GTEN Train	Perhaps the easiest way to use the GTEN functionality is to configure the hardware by sending an XML string using the MCF format described above. The following example demonstrates sending a GTEN Train with the following parameters to the amplifier: 1. **dcWaveform**: The GTEN Train will perform direct current neuromodulation. Notice that the block specifies the id of the waveform to use. 2. **Target electrodes**: Electrodes 1 and 256 will be used for neuromodulation—they are zero-indexed in the GTEN Train format. 3. **Target electrode amplitudes**: 200 uA will be injected into channel 1 and -200 uA will be injected into channel 256. `(sendCommand cmd_GTENSetTrain 0 0 <?xml><train><waveforms><dcWaveform id="0" /></waveforms><blocks> <block id="0"><duration_ms>10000</duration_ms><waveformId>0</waveformId><spatialPattern><electrode channelID="0" amplitude="200" /><electrode channelID="255" amplitude="-200" /></spatialPattern></block> </blocks></train></xml>)`

Feature/Function	Command/Description
Preloading GTEN Waveforms	Because loading the waveforms can take significantly more time than loading the blocks, there are cases where it is advantageous to preload one or more GTEN Waveforms that can be referenced later by GTEN Blocks that are specified later. In particular, if the desired waveform(s) is/are known ahead of time with the targeting information to be determined by the real-time analysis of incoming EEG, you might minimize the delay before the delivery of current can be initialized by preloading the GTEN Waveform(s) prior to determining the targets, then load the GTEN Blocks when this determination is made. Similarly (or in conjunction with the above), if there are a known set of waveforms that might be appropriate as a response to some analysis of the EEG, it is possible to preload all of those waveforms and select from among them (by specifying the appropriate waveformId) when specifying the blocks for the neuromodulation. To load waveforms, you extract the <waveforms> element from the GTEN Train as shown below: `(sendCommand cmd_GTENSetWaveforms 0 0 <?xml><waveforms><dcWaveform id="0" /></waveforms></xml>)`
Loading GTEN Blocks	In order to begin neuromodulation, once one or more waveforms have been loaded, you must load one or more GTEN Blocks referencing the loaded waveforms. Similarly to preloading the GTEN Waveforms, to load GTEN Blocks, extract the <blocks> element from the GTEN Train as shown below: `(sendCommand cmd_GTENSetBlocks 0 0 <?xml><blocks><block id="0"><duration_ms>10000</duration_ms> <waveformId>0</waveformId><spatialPattern><electrode channelID="0" amplitude="200" /><electrode channelID="255" amplitude="-200" /></spatialPattern></block></blocks></xml>)`
Starting Neuromodulation	Once both the GTEN Waveforms and GTEN Blocks have been loaded using one of the techniques described above, the system is armed and ready to be started. `(sendCommand cmd_GTENStartTrain 0 0 1)`
Stopping Neuromodulation	If neuromodulation has been started and is currently running, it is possible to stop the GTEN Train by sending an abort command. `(sendCommand cmd_GTENAbortTrain 0 0 1)` Note that when a running GTEN Train has been aborted, it cannot be restarted without at least reloading the blocks.

Putting It All Together

A minimal GTEN session, as described above, might look like the following.

This session initializes the amplifier and starts acquisition, sets GTEN Waveforms and GTEN Blocks, then starts and (after an unspecified amount of time) aborts the neuromodulation train. Finally, EEG acquisition is stopped and the system is deinitialized.

```
(sendCommand cmd_SetPower 0 0 1)
(sendCommand_return (status complete))
(sendCommand cmd_GTENSetWaveforms 0 0 <?xml><waveforms><dcWaveform id="0" /></waveforms></xml>)
(sendCommand_return (status complete))
(sendCommand cmd_GTENSetBlocks 0 0 <?xml><blocks><block
id="0"><duration_ms>100000</duration_ms><waveformId>0</waveformId><spatialPattern><electrode channelID="0" amplitude="200"
/><electrode channelID="255" amplitude="-200" /></spatialPattern></block></blocks></xml>)
(sendCommand_return (status complete))
(sendCommand cmd_GTENStartTrain 0 0 1)
(sendCommand_return (status complete))
(sendCommand cmd_GTENAbortTrain 0 0 1)
(sendCommand_return (status complete))
(sendCommand cmd_Stop 0 0 1)
(sendCommand_return (status complete))
(sendCommand cmd_SetPower 0 0 1)
(sendCommand_return (status complete))
```

Monitoring TR for Instantaneous GTEN State Information

In order to facilitate synchronization between a client PC and the GTEN nueromodulation subsystem, some state information is inserted into the data packets, along with the EEG. The real-time synchronization signals are inserted into the TR byte (see Appendix 1, "Packet Format 2"). The following are the signals as implemented:

1. 251 (**1111 1011**): **Injecting Current**. When TR&4 is zero, current is being injected. This is useful to know when attempting to reconstruct the signal on the injection channels, as it indicates when the scaling described in the "Compliance Voltage" section should be applied.
2. 239 (**1110 1111**): **Block Start**. A GTEN neuromodulation session is composed of one or more blocks. Once a block is started, it will be synchronous with the sampling rate of the amplifier (1000 samples per second).
3. 229 (**1110 0101**): **Overcurrent Alarm**: This code is produced when the system experiences an overcurrent situation and enters an alarm state. By the time this signal is received, the current injection will have ceased and an audio alarm will have begun sounding.

Compliance Voltage

The compliance voltage of a current source or sink is measured via the associated EEG channel input. The EEG channel input is scaled with a resistor divider network consisting of a 2 M Ohm input resistor and 10 K shunt yielding a divide by 201. This divider scales the EEG input range to +/- 67 volts that is then limited to +/-15 volts due to ESD protection circuitry. The divider also exhibits a burden current of 0.5 uA/volt to the associated current source or sink. Being that the REF signal is subtracted from the EEG channel at the Instrument Amp, the REF is added back after acquisition to make the Compliance Voltage measurement single ended. The Compliance

Voltage measurement is used to provide electrode impedance feedback to the user to determine if it is in range to deliver the desired current.

The following formula, then, must be implemented in order to measure compliance voltage on injection channels:

```
injection_channel = (injection_channel + ref_channel) * 201
```

There are a couple of issues to be aware of:

1. This scaling is only appropriate during the injection period. A pulsed waveform, then, should only be scaled by the above formula during the ON period of the pulse (this can be achieved by watching the state of the TR byte as described in "Monitoring TR for Instantaneous GTEN State Information" section).
2. Signal reconstruction / compliance voltage measurement depends upon the REF channel. This means that if anything affects the REF channel (such as high impedance), the result for all injection channels will be affected.

Source / Sink Aggregate Current

The source and sink aggregate current is captured in bytes 3 and 4 of the reserved array within the data packet. See "Reading the Data Stream" for Packet Format 2. The signal for both source and sink aggregate current is rectified and is therefore always positive, reflecting the absolute amplitude of the injection.

Arbitrary Waveform Frequency Specification

The GTEN system implements an arbitrary waveform generator that allows a variety of different periodic AC waveforms to be generated and executed. This is accomplished by specifying the actual sample values (in the range [-1.0,1.0]) that describe the one period of the AC protocol along with parameters specifying the frequency. Due to constraints introduced by the need to maintain synchronization between the waveform generator and EEG collection, the user must specify the desired frequency in a fashion the system is able to interpret.

Hardware Considerations

The GTEN system includes a microcontroller slaved to the system clock that drives sample collection to ensure that sample generation and sample collection remain in sync. To achieve a specific frequency, the system clock needs to be divided into segments that represent the duration of time each sample is to be held in order to generate the number of periods in one second as described by the frequency.

The system clock speed of 30.72 Mhz means that 30,720,000 ticks are produced every second. To produce a 1 Hz sine wave using all 1024 available samples, the number of ticks per second need to be divided into 1024 equal segments. 30,720,000 divided by 1024 is 30,000, so each sample in the specified waveform will be held for 30,000 ticks of the system clock.

This doesn't always work so nicely: at 7 Hz, for instance, with 1024 samples the division (30,720,000 divided by 7 times 1024) produces a sample duration of 4285.714285714285. It is not possible to further divide the ticks of the clock, so this division must result in an integer value to be valid.

There are two components that can be adjusted in order to produce a waveform with a desired frequency: the number of periods per second and the number of samples per period. In the case of 7 Hz, as 30,720,000 is not evenly divisible by 7, no variation of the number of samples can make the formula work. This is not always the case: for 32 Hz, 30,720, 000 can be evenly divided by 32 and yields the result of 960,000 ticks per period. 960,000, though, cannot be evenly divided by 1024 (the maximum number of samples supported per period). Instead, the 960,000 ticks allotted to the period can be evenly divided among 1000 samples and successfully yields a true 32 Hz signal.

Thus, to determine whether a frequency can be generated by the system, it is necessary to divide the system clock by the desired frequency. The result of this division is the ticks per period. If the result of the previous calculation is an integer value, the next step is to divide the ticks per period by the number of samples in order to achieve the ticks per sample. These formulas are listed below:

$$ticks_{period} = \frac{30{,}720{,}000}{frequency}$$

$$ticks_{sample} = \frac{ticks_{period}}{samplecount}$$

Both $ticks_{period}$ and $ticks_{sample}$ must be whole integer values for the system to be capable of producing the desired frequency.

Approaches

There are two approaches to generating an arbitrary waveform at a desired frequency: one can adjust the frequency parameters and number of samples or generate the samples for a waveform of a particular frequency at a constant sample rate.

ADJUSTING PARAMETERS

The first approach to specifying the frequency is to adjust the parameters for the frequencyNumerator and the frequencyDenominator, along with the number of samples, to achieve a close approximation of the desired frequency. The advantage of this approach is that it allows the full 1024 samples (in most cases) to be used to define the waveform. As described previously, the drawback is that certain frequencies can only be approximated by this method.

For this method, a determination of the best approximation of the target frequency supported by the amplifier must be made first. If the target frequency is an integer value (or 0.5 Hz) less than or equal to 200 Hz, the necessary parameters for the nearest approximation are included in "Parameters of Common Frequencies" table. For other cases, the parameters can be found by applying formulas from "Hardware Considerations" section.

ADJUSTING SAMPLES

The second approach to specifying the frequency for a waveform is to generate the samples for one second of a waveform as informed by the frequency and specify a frequency of 1 Hz (sampleNumerator =1, sampleDenominator = 1) for the waveform.

In this case, generating a 7 Hz waveform would involve generating a set of 1024 samples representing 7 periods. (Of course, defining 7 periods of a signal within 1024 samples presents its own difficulties as 1024 is not evenly divisible by 7.) The main disadvantage of this method is that the higher the frequency of the desired waveform the fewer samples available to represent it. As an example, each period of a 200 Hz sine wave would have to be represented by about 5 samples, which is a pretty coarse sine wave.

Parameters of Common Frequencies

Ideal Frequency	Actual Frequency	Required Samples	Frequency Numerator	Frequency Denominator
0.5	0.5	1024	1000	2000
1	1	1024	1000	1000
2	2	1024	2000	1000
3	3	1024	3000	1000
4	4	1024	4000	1000
5	5	1024	5000	1000
6	6	1024	6000	1000
7	6.99	1024	7000	1001
8	8	1024	8000	1000
9	8.98	1024	9000	1002
10	10	1024	10000	1000
11	11.01	1024	12000	1090
12	12	1024	12000	1000
13	12.99	1024	13000	1001
14	14.01	1024	15000	1071
15	15	1024	15000	1000
16	16	1024	16000	1000
17	17	1024	18000	1059
18	17.96	1024	18000	1002
19	19.01	1024	20000	1052
20	20	1024	20000	1000
21	20.98	1024	21000	1001
22	21.98	1024	22000	1001
23	22.99	1024	24000	1044
24	24	1024	24000	1000
25	25	1024	25000	1000
26	25.97	1024	26000	1001
27	27.03	1024	28000	1036
28	28.01	1024	30000	1071
29	29.01	1024	30000	1034
30	30	1024	30000	1000
31	30.93	1024	33000	1067
32	32	1000	32000	1000
33	32.97	1024	33000	1001
34	34.01	1024	35000	1029
35	34.97	1024	35000	1001
36	35.93	1024	36000	1002
37	37.04	1024	38000	1026
38	38.02	1024	40000	1052
39	38.96	1024	39000	1001
40	40	1024	40000	1000
41	40.98	1024	45000	1098
42	42.02	1024	45000	1071

Ideal Frequency	Actual Frequency	Required Samples	Frequency Numerator	Frequency Denominator
43	42.98	1024	45000	1047
44	43.99	1024	45000	1023
45	44.91	1024	45000	1002
46	46.08	1024	50000	1085
47	46.95	1024	50000	1065
48	48	1024	48000	1000
49	49.02	1024	50000	1020
50	50	1024	50000	1000
51	51.02	1024	55000	1078
52	52.08	1024	55000	1056
53	53.1	1024	54000	1017
54	54.05	1024	56000	1036
55	55.05	1024	60000	1090
56	55.97	1024	60000	1072
57	57.03	1024	60000	1052
58	58.03	1024	60000	1034
59	59.06	1024	60000	1016
60	60	1024	60000	1000
61	60.98	1024	65000	1066
62	61.86	1024	66000	1067
63	63.16	1024	66000	1045
64	64	1000	64000	1000
65	64.94	1024	65000	1001
66	65.93	1024	66000	1001
67	67.11	1024	70000	1043
68	68.03	1024	70000	1029
69	68.97	1024	70000	1015
70	69.93	1024	70000	1001
71	71.09	1024	75000	1055
72	72.12	1024	75000	1040
73	72.99	1024	80000	1096
74	74.07	1024	76000	1026
75	75	1024	75000	1000
76	75.95	1024	78000	1027
77	76.92	1024	77000	1001
78	77.92	1024	78000	1001
79	78.95	1024	81000	1026
80	80	1024	80000	1000
81	81.08	1024	84000	1036
82	81.97	1024	85000	1037
83	83.1	1024	90000	1083
84	84.03	1024	90000	1071
85	84.99	1024	90000	1059
86	85.96	1024	90000	1047
87	86.96	1024	88000	1012
88	87.98	1024	90000	1023
89	89.02	1024	90000	1011
90	89.82	1024	90000	1002
91	90.91	1024	91000	1001
92	91.74	1024	100000	1090
93	92.59	1024	95000	1026

Ideal Frequency	Actual Frequency	Required Samples	Frequency Numerator	Frequency Denominator
94	93.75	1024	96000	1024
95	95.24	1024	104000	1092
96	96	1000	96000	1000
97	97.09	1024	100000	1030
98	98.04	1024	100000	1020
99	99.01	1024	100000	1010
100	100	1024	100000	1000
101	101.01	1024	110000	1089
102	102.04	1024	110000	1078
103	103.09	1024	110000	1067
104	104.17	1024	110000	1056
105	104.9	1024	105000	1001
106	106.38	1024	110000	1034
107	107.14	1024	108000	1008
108	107.53	1024	110000	1023
109	109.09	1024	114000	1045
110	109.89	1024	110000	1001
111	111.11	1024	116000	1044
112	111.94	1024	120000	1072
113	113.21	1024	120000	1060
114	114.07	1024	120000	1052
115	114.94	1024	120000	1044
116	115.83	1024	120000	1036
117	117.19	1024	120000	1024
118	118.11	1024	120000	1016
119	119.05	1024	125000	1050
120	120	1024	120000	1000
121	121.44	1020	128000	1054
122	121.95	1024	125000	1025
123	122.95	1024	135000	1098
124	123.97	1024	135000	1089
125	125	1024	125000	1000
126	126.05	1024	135000	1071
127	127.12	1024	135000	1062
128	128	1000	128000	1000
129	129.31	1024	135000	1044
130	129.87	1024	130000	1001
131	131.15	976	144000	1098
132	131.58	1024	135000	1026
133	132.74	1024	135000	1017
134	133.93	1024	135000	1008
135	135.14	1024	140000	1036
136	136.36	1024	141000	1034
137	136.99	1024	140000	1022
138	138.25	1024	150000	1085
139	138.89	1024	140000	1008
140	140.19	1024	150000	1070
141	140.85	1024	150000	1065
142	142.18	1024	150000	1055
143	142.86	1024	143000	1001
144	144.23	1024	150000	1040

Ideal Frequency	Actual Frequency	Required Samples	Frequency Numerator	Frequency Denominator
145	144.93	1024	150000	1035
146	146.34	1024	150000	1025
147	147.06	1024	150000	1020
148	147.78	1024	150000	1015
149	149.25	1024	160000	1072
150	150	1024	150000	1000
151	150.94	960	152000	1007
152	151.52	1024	155000	1023
153	153.06	1024	165000	1078
154	153.85	1024	154000	1001
155	154.64	1024	165000	1067
156	156.25	1024	160000	1024
157	156.56	1022	160000	1022
158	157.89	1024	159000	1007
159	158.73	1024	160000	1008
160	160	1000	160000	1000
161	161.29	1024	170000	1054
162	162.16	1024	168000	1036
163	163.04	1024	165000	1012
164	163.93	1024	170000	1037
165	164.84	1024	165000	1001
166	165.75	1024	180000	1086
167	166.67	1024	167000	1002
168	167.6	1024	180000	1074
169	168.54	1024	180000	1068
170	170.45	1024	180000	1056
171	171.43	1024	180000	1050
172	172.41	1024	180000	1044
173	173.41	1024	180000	1038
174	174.42	1024	180000	1032
175	175.44	1024	180000	1026
176	176.47	1024	180000	1020
177	176.96	992	192000	1085
178	177.51	1024	180000	1014
179	178.57	1024	180000	1008
180	179.64	1024	180000	1002
181	180.72	1024	195000	1079
182	181.82	1024	182000	1001
183	182.93	1024	195000	1066
184	184.33	992	200000	1085
185	185.19	1024	200000	1080
186	186.05	960	192000	1032
187	187.32	1000	192000	1025
188	187.68	1023	192000	1023
189	188.68	1024	190000	1007
190	189.87	1024	195000	1027
191	191.08	1024	210000	1099
192	192	1000	192000	1000
193	192.77	996	208000	1079
194	193.55	1024	198000	1023
195	194.81	1024	195000	1001

Ideal Frequency	Actual Frequency	Required Samples	Frequency Numerator	Frequency Denominator
196	196.08	1024	200000	1020
197	197.37	1024	210000	1064
198	197.53	972	208000	1053
199	198.68	1024	210000	1057
200	200	1024	200000	1000
201	201.34	1024	210000	1043
202	201.68	952	216000	1071
203	202.7	1024	210000	1036
204	204.08	1024	220000	1078
205	205.48	1024	210000	1022
206	206.45	992	224000	1085
207	206.9	1024	210000	1015
208	208.33	1024	215000	1032
209	209.15	1020	224000	1071
210	209.79	1024	210000	1001
211	211.27	1024	225000	1065
212	211.92	960	224000	1057
213	212.77	1024	230000	1081
214	214.29	1024	216000	1008
215	215.05	992	220000	1023
216	216.22	960	224000	1036
217	217.39	1024	230000	1058
218	217.69	1008	224000	1029
219	218.98	1024	240000	1096
220	220.39	968	240000	1089
221	220.59	1024	225000	1020
222	222.22	1024	232000	1044
223	222.63	896	240000	1078
224	223.88	1024	225000	1005
225	225.35	960	240000	1065
226	225.56	1024	240000	1064
227	227.27	1024	235000	1034
228	227.7	992	240000	1054
229	229.01	1024	240000	1048
230	229.89	928	240000	1044
231	230.77	1024	231000	1001
232	231.88	960	240000	1035
233	232.56	1024	250000	1075
234	234.38	1024	255000	1088
235	234.6	1023	240000	1023
236	236.22	1024	240000	1016
237	236.69	1014	240000	1014
238	238.1	1024	250000	1050
239	239.03	1020	256000	1071
240	240	1024	240000	1000
241	240.96	996	260000	1079
242	241.94	1024	255000	1054
243	242.88	1020	256000	1054
244	243.9	1024	250000	1025
245	244.9	980	252000	1029
246	245.9	1024	255000	1037

Ideal Frequency	Actual Frequency	Required Samples	Frequency Numerator	Frequency Denominator
247	246.87	1020	256000	1037
248	247.93	1024	270000	1089
249	248.79	980	256000	1029
250	250	1024	250000	1000

End-User Software License

The Amp Server Pro SDK is licensed to the end user for their own internal research use under the terms of the Magstim Inc Amp Server Pro SDK license. No use of the Amp Server Pro SDK to produce a product or service that will be resold, licensed, transferred, or shared is permitted by the end-user license. No clinical use of this product is permitted under this license.

Amp Server Pro SDK
End-User Software License

Read the full license before downloading the software.

THIS INSTRUMENT IS NOT INTENDED FOR USE IN THE DIAGNOSIS OR TREATMENT OF ANY DISEASE OR CONDITION. IT IS A SCIENTIFIC RESEARCH INSTRUMENT DESIGNED FOR PERFORMING MEASUREMENTS AND ACQUIRING DATA FOR NEUROPHYSIOLOGICAL RESEARCH. MAGSTIM INC MAKES NO REPRESENTATION OF THE SUITABILITY OF THE INSTRUMENT FOR ANY PARTICULAR RESEARCH STUDY.

For questions or additional assistance, please contact us at:

Magstim Inc
78 Centennial Loop, Suite C
Eugene, OR 97401 USA

+1.541.687.7962 (tel)
+1.541.687.7963 (fax)

support@magstimegi.com
www.egi.com